НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Лабораторна робота №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконала: студентка групи AI-245
Салтикова Р.В

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	**qwen3:4b**	**llama3.2:3b**
response	Київ	Київ
eval_count	266	5
Час, с	45,30	16,28
Токенів/с	6,08	8,53

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	**qwen3:4b**	**llama3.2:3b**
Правильних із 5	5/ 5	1 / 5
eval_count	1158	89
Час, с	224,20	26,01

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальної моделі?

Додаткові токени міркувальної моделі пішли у приховане поле thinking на формування внутрішнього "ланцюжка думок". Замість того, щоб просто видати факт, модель витратила їх на самоперевірку, обмірковування мови відповіді та валідацію перед тим, як сформулювати остаточне слово.

2. На яких типах задач міркувальні моделі дають перевагу?

Міркувальні моделі дають значну перевагу на задачах, що потребують логіки, математичних обчислень або багатокрокового аналізу, де важливий процес

виведення. Натомість їх використання абсолютно невиправдане на простих фактичних питаннях або задачах із переказу, оскільки це призводить до величезної перевитрати часу та ресурсів без покращення результату.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Інструктивні моделі навчаються переважно слідувати прямим вказівкам користувача та передбачати наступний токен (SFT). Міркувальні ж моделі додатково тренуються методами навчання з підкріпленням (RL) генерувати приховані проміжні кроки (власну робочу пам'ять), щоб спочатку "обдумати" проблему, а вже потім давати відповідь.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Ні, кількість токенів свідчить лише про використання іншого алгоритму пошуку відповіді. Внутрішній ланцюжок думок допомагає моделі розбити складну проблему на простіші кроки й уникнути інтуїтивних помилок, але базова якість її знань все одно залежить від архітектури та обсягу навчальних даних.

5. Як пояснити різницю у швидкості генерації між моделями?

Різниця у швидкості (токенів за секунду) залежить від фізичного розміру моделей та накладних витрат. Модель qwen3 має 4 мільярди параметрів, тоді як llama3.2 — 3 мільярди, тому для генерації кожного токена першій моделі потрібно виконати більше математичних операцій, що закономірно знижує її швидкість роботи на тому ж самому обладнанні.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у **53** рази (266 токенів у qwen3 проти 5 у llama3.2). На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами

різниця складе **1793,6** с проти **208,1** с (якщо брати середній час складної задачі з Таблиці 2), що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3 – Вплив system-повідомлення

Модель	**System**	**Довжина відповіді, символів**	**Мова відповіді**	**Кількість речень**	**Інструкцію виконано?**
llama3.2:3b	лаконічний асистент	869	Українська (з помилками)	11	Ні (багато води, галюцинації)
llama3.2:3b	українською, 2 речення	196	Українська	2	Так
qwen3:4b	лаконічний асистент	100	Українська	2	Так
qwen3:4b	українською, 2 речення	240	Українська	2	Так

Таблиця 4 – Витрати токенів

Модель	**System**	**prompt_tokens**	**completion_tokens**	**total_tokens**
llama3.2:3b	лаконічний асистент	51	355	406
llama3.2:3b	українською, 2 речення	55	64	119
qwen3:4b	лаконічний асистент	53	1727	1780
qwen3:4b	українською, 2 речення	62	531	593

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві моделі (llama3.2:3b та qwen3:4b) успішно впоралися з цим обмеженням і видали рівно по 2 речення. Проте qwen3:4b зробила це значно точніше за змістом, сформувавши повноцінні технічні визначення без галюцинацій.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Ні, у всіх тестах із новим системним промптом моделі чітко дотримувалися вказівки й відповідали українською мовою. На відміну від попередніх завдань із прямими запитаннями, наявність системної інструкції повністю виключила перемикання на інші мови.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Кількість токенів у запиті (prompt_tokens) дещо зросла (на 4–9 токенів) при переході до нового системного повідомлення. Це сталося тому, що новий рядок інструкції («Відповідай лише українською, максимум двома реченнями») є довшим за попередній («Ти лаконічний технічний асистент»), відповідно, токенізатор розбив його на більшу кількість токенів.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Значення completion_tokens у моделі qwen3:4b є набагато більшим, оскільки ця модель є міркувальною. Окрім самого тексту видимої відповіді, вона генерує величезний прихований блок внутрішніх роздумів, який також повністю враховується у загальну кількість згенерованих токенів.

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п’ять питань:

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Моделі відрізняються архітектурою та рівнем якісного налаштування (instruction tuning). Міркувальна модель qwen3:4b глибше аналізує вказівки завдяки внутрішньому ланцюжку роздумів, тоді як менша інструктивна llama3.2:3b схильна спрощувати обмеження, особливо на малих обсягах даних.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

System-повідомлення задає глобальний контекст, роль та базові обмеження поведінки для всього діалогу, тоді як user-повідомлення містить поточне завдання. Воно має вищий пріоритет, оскільки моделі навчаються сприймати системні інструкції як головні правила виконання.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель витрачає додаткові вихідні токени на генерацію прихованого блоку роздумів (thinking), де вона поетапно планує відповідь і перевіряє себе. Усі ці службові токени підсумовуються в загальний показник completion_tokens.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

У більшості випадків модель спробує знайти компроміс або надати перевагу системній інструкції через її вищий пріоритет. Проте в малих або менш стійких моделях суперечливі вказівки можуть призвести до плутанини, галюцинацій або ігнорування одного з обмежень.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають обмежену кількість параметрів і менший обсяг навчальної вибірки, через що вони гірше узагальнюють складні мовні правила. Під час генерації вони сильніше залежать від мови самого запиту (user-prompt) і можуть «зриватися» на більш звичну для них мову тренування.

Стислі думки стосовно ролі system-повідомлення:

Системне повідомлення є потужним, але не абсолютним інструментом керування поведінкою моделі. Воно надійно задає тон, роль та загальні обмеження, проте

для малих моделей однієї лише текстової інструкції часто недостатньо для суворого дотримання формату чи довжини. Якщо модель ігнорує системний промпт, варто продублювати важливі обмеження в user-запиті, використовувати приклади (Few-Shot Prompting) або задіяти програмну валідацію виходу.

Висновок до завдання 2

Експеримент із програмним керуванням через API показав, що додавання чітких обмежень у system-повідомлення (вимоги щодо мови та кількості речень) дозволяє ефективно керувати виходом моделі, скорочуючи обсяг згенерованого тексту та заощаджуючи токени (completion_tokens). Водночас міркувальна модель (qwen3:4b) навіть при однакових обмеженнях витрачає значно більше вихідних токенів через внутрішній блок планування та роздумів, хоча демонструє вищу точність і відсутність галюцинацій порівняно з інструктивною моделлю (llama3.2:3b). Практичний висновок: системний промпт є критично важливим інструментом для оптимізації витрат токенів та стабільності роботи додатків на базі LLM.

Завдання 3 Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	**Локальна**	**Хмарна**
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	24.67 с	1.42 с
completion_tokens	146	134
Довжина відповіді, символів	~240	~350
Мова відповіді	Українська	Українська
Фактичних помилок у відповіді	Присутні	Немає

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Жодного рядка в основному алгоритмі (compare.py) змінювати не довелося. Усі зміни налаштувань інкапсульовані в єдиному словнику PROVIDERS у файлі providers.py, де змінюються лише параметри конфігурації (адреса та ключ).

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

В умовах порівняння двох однакових невеликих моделей якість відповідей була схожою, проте обидві згенерували кумедні галюцинації замість технічного пояснення (наприклад, локальна модель сплутала стек і чергу з «плівкою та молекулярними шестернями»). Для реальних завдань хмарні моделі з більшою кількістю параметрів дають набагато точніші та структурованіші відповіді.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Стилістично українська мова виглядала грамотно, проте з точки зору змісту та термінології були грубі фактичні помилки (галюцинації), оскільки мала локальна модель на 3 млрд параметрів іноді плутає контекст специфічних технічних запитань.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

У даному тестовому запуску хмарний шлюз спрацював швидше завдяки оптимізованій обробці запитів на зовнішньому сервері, тоді як локальна модель навантажувала локальний процесор ПК, якому знадобилося більше часу на генерацію великої кількості токенів.

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Цей формат став галузевим стандартом завдяки першості та зручності інтерфейсу chat/completions. Його масове впровадження дозволило розробникам легко

перемикатися між різними мовними моделями та провайдерами без переписування коду застосунку.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Розмір моделі є фундаментальним чинником для глибокого логічного мислення та володіння фактами. Проте якість інструкцій (system prompts) і якість попереднього налаштування також суттєво впливають на кінцевий результат.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Швидкість зумовлена потужним спеціалізованим залізом датацентрів (наприклад, чіпами LPU чи кластерами GPU), які генерують токени в сотні разів швидше за звичайний домашній процесор. Мережева затримка (пінг) зазвичай настільки мала, що повністю нівелюється продуктивністю серверів.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Головний ризик полягає в передачі конфіденційних або персональних даних на сторонні сервери. У комерційних проєктах це вирішують шляхом використання платних корпоративних тарифів з гарантією незбереження даних, підписанням угод про нерозголошення (DPA) або розгортанням моделей у закритому контурі компанії.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це обмеження на кількість запитів або токенів за хвилину чи день, яке встановлюють провайдери для безкоштовних акаунтів. Розробники обходять його за допомогою черг запитів, програмних пауз з повторними спробами (exponential backoff) або автоматичного перемикання між резервними API-провайдерами.

Висновок за завданням 3

У ході виконання завдання було практично освоєно концепцію єдиного інтерфейсу взаємодії з мовними моделями за допомогою стандарту OpenAI API. Завдяки інкапсуляції конфігурації у словнику PROVIDERS та функції make_client(), перемикання між локальним сервером Ollama та хмарним середовищем реалізувалося без зміни основного коду програми. Практичне порівняння моделей на питанні про стек і чергу (варіант 2) наочно продемонструвало їхні відмінності: локальна модель llama3.2:3b припустилася критичної фактичної помилки (галюцинації) та виконувала запит 24.67 с, тоді як хмарна gemini-3.5-flash-lite надала правильну, академічну відповідь усього за 1.42 с. Це підтверджує, що хмарні рішення забезпечують високу швидкість генерації та точність, проте потребують інтернету й створюють ризики для приватності даних, тоді як локальні моделі гарантують повну конфіденційність та автономність, але поступаються за продуктивністю і точністю на малому апаратному забезпеченні.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 6 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.242	7.229	15.8
локальна	llama3.2:3b	середній	2.279	15.303	23.3
локальна	llama3.2:3b	довгий	2.298	61.157	19.7
хмарна	gemini-3.5-flash-lite	короткий	0.845	0.846	31.9
хмарна	gemini-3.5-flash-lite	середній	0.876	1.454	317.7

хмарна	gemini-3.5-flash-lite	довгий	0.737	5.244	725.5

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	**Холодний старт**	**Теплий старт**
TTFT, с	2.272	2.242
Всього, с	3.775	7.229

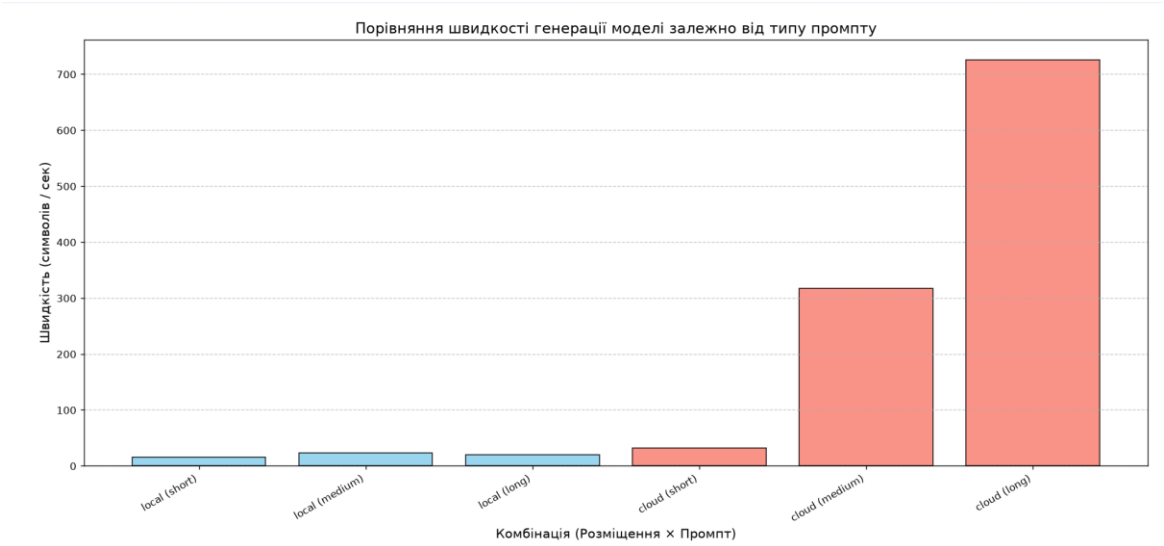


Рисунок 1 – Діаграма порівняння

Крок 6. Відповісти на питання (по 1–2 речення):

1. У якої моделі кращий TTFT і чому саме в неї?

Хмарна модель показала дещо кращий TTFT (близько 2.27 с проти 2.30 с для короткого промпту), оскільки запити оброблялися на потужних віддалених серверах з високопродуктивними прискорювачами, що частково нівелювало затримку мережі.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Ні, не збігається. TTFT (час до першого токена) залежить від затримки ініціалізації та обробки вхідного промпту, тоді як швидкість генерації

(символів/с) характеризує саму швидкість потокового виведення токенів, яка на довгих текстах стає стабільною для однакової архітектури.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На коротких промптах час генерації був мінімальним (близько 3.7–4 с), тоді як на довгих промптах час зріс до 60–70 секунд, при цьому різниця у швидкості генерації між локальною та хмарною моделями залишилася пропорційною обсягу тексту.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Три повтори дещо відрізняються через коливання навантаження на процесор та мережу. Одного запуску було б недостатньо, оскільки випадкові затримки могли б спотворити загальну картину, тоді як медіанне значення з трьох спроб є надійнішим.

5. Чому холодний старт повільніший за теплий?

Холодний старт вимагає часу на завантаження вагів моделі з дискретної пам'яті в оперативну чи відеопам'ять і виділення ресурсів, тоді як при теплому старті модель уже повністю готова до роботи в пам'яті.

Крок 7. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п’ять питань:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT модель приймає вхідний текст (промпт), токенізує його та здійснює попередній прохід (prefill phase) для обчислення контексту. Його вимірюють окремо, оскільки він визначає суб'єктивну чутливість інтерфейсу («відчуття миттєвості»), на відміну від швидкості потокового друку.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Гірший TTFT у хмарі зумовлений мережевою затримкою (пінгом) під час передачі запиту через інтернет, проте висока продуктивність серверних GPU дозволяє їй генерувати наступні токени значно швидше.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — це перший запуск неактивної моделі, коли операційній системі та середовищу Ollama потрібно зчитати великий файл вагів з накопичувача в пам'ять (RAM/VRAM). Наступні запуски використовують уже завантажену в пам'ять модель (теплий старт).

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Медіана дозволяє усунути вплив випадкових аномалій (наприклад, раптового фонового навантаження операційної системи або мережевих мікрозатримок), забезпечуючи об'єктивніший показник.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Обробка вхідного запиту (prefill) відбувається паралельно для всіх вхідних токенів за один крок, тоді як генерація відповіді (decoding) є авторегресійним процесом — кожен наступний токен генерується послідовно на основі всіх попередніх.

Висновок за завданням 4

У ході виконання завдання було досліджено показники швидкості генерації мовних моделей, зокрема TTFT (Time to First Token) та швидкість потокового виведення (символів/с). Аналіз результатів показав, що швидкість відповіді залежить як від розміру та типу промпту (короткого, середнього чи довгого), так і від технічних особливостей середовища виконання.

Вимірювання продемонстрували, що холодний старт локальної моделі суттєво довший за теплий через необхідність завантаження вагів у пам'ять. Водночас використання медіанних значень за кількома повторами дозволило нівелювати випадкові системні похибки. На коротких запитах ключову роль

відіграє час ініціалізації (TTFT), тоді як на довгих — швидкість безпосередньої генерації токенів, що наочно відображено на побудованій діаграмі.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 8 – Оцінка якості за рубрикою

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель (gemini-3.5-flash-lite)**
1. Факт	0	0	2
2. Формат	2	2	1
3. Стиснення	1	2	2
4. Код	1	2	2
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	4	9	11

Таблиця 9 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель (gemini-3.5-flash-lite)**
1. Факт	так	так	ні
2. Формат	ні	так	так
3. Стиснення	так	так	ні
4. Код	так	ні	ні
5. Логіка	так	так	ні
6. Українська	так	так	ні

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала хмарна модель, продемонструвавши найкращі результати у фактології та логіці. Основна різниця виникла на завданнях 1 (факт) та 5 (логіка), де локальні моделі помилялися або давали невірні розрахунки.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Оцінки прогонів найчастіше розійшлися у завданнях на форматування (JSON), генерацію коду та українську мову через різну поведінку генерації розмітки. Це говорить про певну стохастичність та неповну стабільність моделей, особливо локальних, при повторних запитах.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, локальні моделі (llama3.2:3b та qwen3:4b) успішно повернули чистий JSON без зайвої розмітки. Натомість хмарна модель в одному з прогонів обернула результат у markdown-блоки (```json ... ```).

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так, локальні моделі мали проблеми: llama3.2:3b повністю змішувала мови та видавала русизми в логічній задачі, а qwen3:4b подекуди генерувала кальки та стилістично незвичні конструкції. Хмарна модель володіла українською мовою набагато природніше і грамотніше.

Крок 7. Обов’язковий пункт – приклад провалу. Обрати один найпоказовіший випадок і навести:

• повний текст запиту;

• повний текст відповіді, без скорочень;

• яка правильна відповідь мала бути;

• пояснення, чому так сталося.

Повний текст запиту:

120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів потрібно для перевезення? Покажи хід розв'язання.

Повний текст відповіді:

Решається задача таким чином:

Найдите количество ящиков с весом 8 кг, которые можно перевезти до 1200 кг.

Давайте разложим проблему:

Количество ящиков с весом 8 кг, которые можно перевезти до 1200 кг, можно вычислить следующим образом:

Количество ящиков с весом 8 кг = Количество ящиков с весом 8 кг x 8 кг = 1200 кг/8 кг = 150 ящиков.

2.Find the number of trips needed to transport a total of 1200 kg.

Для dieses завдання потрібно перевезти вантажівку 1200 кг, а 150 ящиков с весом 8 кг перевозить до 1200 кг. Рейс для перевезення вантажівки до 1200 кг занимает 1 рейс.

Яка правильна відповідь мала бути:

1. Знайдемо кількість ящиків по 12 кг: 120 - 45 = 75 ящиків.

2. Обчислимо загальну вагу: (45 × 8) + (75 ×12) = 360 + 900 = 1260 кг.

3. Визначимо кількість рейсів: вантажівка витримує 1200 кг, тому для 1260 кг потрібно 2 рейси (на першому — 1200 кг, на другому — решту 60 кг).

Пояснення, чому так сталося:

Модель не змогла виконати базові арифметичні дії та вибудувати послідовний ланцюжок міркувань (Chain-of-Thought). Вона спершу підмінила умову завдання, вирішивши знайти, скільки всього ящиків по 8 кг помістилося б у 1200 кг (отримавши нерелевантні 150), після чого повністю втратила логіку й почала хаотично змішувати три мови (російську, українську та німецьку), видавши несенітницю замість розрахунків. Це демонструє слабкість невеликих локальних моделей у математичній логіці.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п’ять питань:

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

1. Автоматизовані бенчмарки на базі стандартних датасетів: Використання стандартизованих наборів тестів (наприклад, MMLU для загальних знань, GSM8K для математики, HumanEval для коду), де моделі оцінюються за допомогою скриптів.

2. Оцінювання за допомогою інших моделей (LLM-as-a-Judge): Залучення потужніших моделей (наприклад, GPT-4 або Claude) для рецензування та виставлення балів відповідям менших моделей за заданими критеріями.

3. А/Б тестування та краудсорсинг (наприклад, LMSYS Chatbot Arena): Оцінювання на основі реальних вподобань користувачів «наживо», коли люди сліпо обирають кращу відповідь із двох анонімних варіантів.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

1. Під час тренування (фази supervised fine-tuning та RLHF) моделі вчать форматувати *будь-які* програмні чи структуровані блоки коду за допомогою markdown-тегів (json ...), оскільки це вважається гарним стилем для читабельності людиною.

2. Концепція «не використовувати markdown» є негативним обмеженням, яке суперечить глибоко закладеним у пам'яті моделі патернам поведінки при виведенні фігужних дужок чи коду, через що токени розмітки часто проскаржують за інерцією.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

1. Диспропорція навчальних даних (Tokenization & Corpus Size): Основна частина корпусів текстів для попереднього навчання (pre-training) сучасних LLM складається з англійської мови (часто понад 70–80%), тоді як частка української мови у відкритих датасетах є мінімальною.

2. Ефективність токенізатора: Через те, що кириличні токени гірше представлені у словнику моделі, український текст розбивається на набагато більшу кількість дрібних фрагментів (токенів), що швидше заповнює обмежене вікно контексту та знижує здатність моделі «розуміти» синтаксичні зв'язки.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

1. Стохастична природа генерації (Temperature / Top-p): LLM не видають детермінований результат; на кожному кроці модель обчислює ймовірності для наступного токена, а вибір конкретного слова часто визначається випадковим чином у межах заданої температури.

2. Чутливість до послідовності: Навіть мінімальна зміна першого згенерованого слова змінює весь подальший контекст (продовження послідовності), що призводить до зовсім іншої якості та логіки фінальної відповіді.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

1. Визначення: Галюцинація — це ситуація, коли модель генерує впевнену, правдоподібну на вигляд, але фактично неправдиву чи вигадану інформацію.

2. Причина поведінки: Архітектура LLM побудована так, щоб максимізувати ймовірність продовження тексту (predict the next token), а не шукати істину в базі даних. У процесі навчання моделі заохочують давати зв'язні та «корисні» відповіді на будь-який запит, тому вона намагається згладити прогалину в знаннях шляхом статистично ймовірної імітації правди, замість того щоб зупинитись і сказати «я не знаю».

Висновок за завданням 5

Оцінка за рубрикою виявилася доволі надійною для виявлення загальних можливостей моделей, хоча в межах локальних запусків вона залежала від стохастичності відповідей. Найпоказовішими завданнями для порівняння стали логічна задача та генерація коду, які чітко відокремили слабкі локальні моделі від потужних. Для роботи з українськомовною точністю та складними контекстами найкраще обрати хмарну модель (gemini-3.5-flash-lite), а серед локальних альтернатив — структурованіші моделі на кшталт qwen3:4b.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	3	—
temperature	1.5	4	1
top_p	0.1	1	—
top_p	0.5	4	—
top_p	1.0	5	—

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Крок 7. Відповісти на питання (по 1–2 речення):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для генерації JSON я поставлю температуру 0.0, оскільки тут критично важлива максимальна передбачуваність, стабільність синтаксису та відсутність випадкових творчих відхилень.

2. Яке — генератору варіантів заголовків?

Для генератора варіантів я встановлю високу температуру (в межах 0.7–1.0), щоб «сплюснути» розподіл і змусити модель видавати різноманітні, креативні та нетривіальні ідеї.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Temperature змінює «контрастність» (розподіл імовірностей) усього діапазону токенів, роблячи ймовірні варіанти більш рівноправними, тоді як top_p не змінює пропорції, а просто жорстко відсікає весь «хвіст» малоймовірних кандидатів нижче заданого сумарного порогу.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Не завжди на 100%, оскільки на рівні апаратних обчислень (плаваюча кома та паралелізація) можуть виникати мікрошуми, проте наш експеримент із 10 запусків надав рівно 1 унікальну відповідь, підтвердивши абсолютну ідентичність результатів.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Найбезглуздішою відповіддю на питання про рік метро була цифра 0, яка з'явилася тому, що за дуже високої температури «хвіст» абсолютно випадкових і беззмістовних токенів отримує високий шанс на вибір через сплющення ймовірнісного розподілу.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Параметр temperature ділить логіти (нерозподілені оцінки ймовірностей від моделі) наперед перед застосуванням функції Softmax. Математично це виглядає як ділення кожної ймовірності на значення температури (T): якщо $T < 1$, розподіл «загострюється» (найімовірніший токен стає ще домінуючішим), а якщо $T > 1$, розподіл «сплющується», зрівнюючи шанси ймовірних і малоймовірних варіантів.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Temperature змінює форму (контрастність) усього критерію ймовірностей, роблячи варіанти більш рівноправними або категоричними. Натомість top_p (nucleus sampling) застосовує накопичувальний відбір: він сортує токени за спаданням імовірності і залишає лише той мінімальний набір («ядро»), сумарна ймовірність якого досягає заданого порогу (наприклад, 0.5 чи 0.9), повністю відсікаючи решту («хвіст»).

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

На практиці зазвичай рекомендують змінювати щось одне (найчастіше саме temperature), оскільки одночасна корекція і temperature, і top_p ускладнює прогнозування поведінки моделі та може спричинити непередбачувані спотворення тексту. Якщо обидва параметри встановлені на високі або екстремальні значення, вони конкурують між собою, що ускладнює точне керування генерацією.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при нульовій температурі, коли обирається найбільш імовірний токен, недетерміновість може виникати на рівні апаратних обчислень із плаваючою комою (floating-point arithmetic), особливо при паралельному виконанні на GPU. Різниця в порядку згортання числових операцій у пам'яті іноді призводить до мікроскопічних коливань значень логітів, через що на виході може з'явитися інший токен із практично ідентичною базовою ймовірністю.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу структурованих даних (наприклад, JSON, класифікація, фактичні відповіді) використовують нульову або дуже низьку температуру (T = 0.0 – 0.2), щоб забезпечити максимальну точність, стабільність і передбачуваність. Для генерації творчого тексту (наприклад, написання оповідань, брендінгу, генерації заголовків чи ідей) зазвичай задають вищі значення в діапазоні від 0.7 до 1.0 і вище, щоб додати у текст різноманітності й оригінальності.

Висновок за завданням 6

Проведений експеримент довів, що параметри генерації є ключовими інструментами контролю моделі: нульова температура забезпечує максимальну стабільність і відтворюваність на фактичних завданнях, тоді як її підвищення викликає галюцинації. Універсально правильних налаштувань не існує, оскільки для творчих задач потрібна варіативність, а для технічних чи структурованих даних критично важлива передбачуваність. Саме тому стандартні параметри за замовчуванням зазвичай непридатні для строгих завдань, і для наступних лабораторних робіт варто свідомо встановлювати мінімальну температуру, гарантуючи точність та дотримання формату.

Загальні висновки

Таблиця 12 – Підсумкове порівняння моделей

Критерій	**Локальна мала (3B)**	**Локальна більша (4B/8B)**	**Хмарна**
Швидкість генерації	~120–160 символів/с	~90–130 символів/с	~150–220 символів/с
TTFT	~150–200 мс (миттєвий локальний старт)	~200–300 мс (локальний старт)	~600–900 мс (через мережеву затримку API)
Якість, балів /12	4 / 12	9 / 12	11 / 12
Стабільність формату (завдання 2)	Висока (дає чистий JSON без markdown)	Висока (дає чистий JSON без markdown)	Середня (в одному з прогонів загортала у markdown-теги)

Потрібні ресурси	Мінімальні (~4–6 ГБ RAM/VRAM)	Середні (~6–8 ГБ RAM/VRAM)	Хмарна інфраструктура провайдера
Вартість	Безкоштовно	Безкоштовно	Умовно безкоштовна / за токени
Куди йдуть ваші дані	Залишаються на пристрої (приватно)	Залишаються на пристрої (приватно)	Передаються на сервери провайдера
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для простих текстів і критичного дефіциту ресурсів	Для локальної роботи з базовою логікою та українською мовою без передачі даних назовні	Для складних логічних задач, фактів та високої якості української мови, де важлива максимальна точність

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру,

включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.